© Guven

Generative Adversarial Networks

```
"What I cannot create, I do not
understand." Richard Feynman
```

Problems and Improvements

🔶 Mode Collapse

The generator finds a small number of samples that deceive the discriminator and, as a result, will not produce any new fake examples. A single observation (a mode) that always deceives the discriminator will map every point in the latent input space to this observation.

🔶 Uninformative Loss

A lack of correlation between the generator loss and image quality will make GAN training difficult to optimize and monitor.

🔶 Hyperparameters

The number of hyperparameters is more than doubled compared to a regular deep NN. As batch normalization, dropout, learning rate, activation layers, convolutional filters, kernel size, striding, batch size, and latent space size must be set. GANs are highly sensitive to very slight changes in hyperparameters.

🔶 Wasserstein GAN

Wasserstein loss function is a better loss metric that correlates with the generator's convergence and sample quality and improves the stability of the optimization process. The discriminator operates like a critic. Reminder,

$$\text{BCE-loss} = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log p_i + (1-y_i)\log(1-p_i)\right) \tag{1}$$

Then,

$$\text{Wasserstein-loss} = -\frac{1}{n}\sum_{i=1}^{n}(y_i p_i) \tag{2}$$

Discriminator (or WGAN critic) loss is computed when comparing predictions for real sample $p_i = D(x_i)$ to the response $y_i = 1$ and predictions for generated sample $p_i = D(G(z_i))$ to the response $y_i = -1$. Therefore, the WGAN critic loss minimization,

$$\min_D \left(\mathbf{E}_{x\sim P_X}[D(x)] - \mathbf{E}_{z\sim P_Z}[D(G(z))]\right) \qquad (3)$$

And WGAN generator loss minimization,

$$\min_G \left(\mathbf{E}_{z\sim P_Z}[D(G(z))]\right) \qquad (4)$$

🔶 Implementation

- Update the loss function
- Remove the sigmoid activation function of the discriminator and make the discriminator a **regressor**
  - Original GAN discriminator is a classifier
- Change real and fake outputs to 1 and -1

🔶 WGAN-GP

- Loss function includes a gradient penalty term based on multiple interpolated sample
- Skip clipping the weights of the critic network
- Skip batch normalization layers for the critic network

🔶 Performance Evaluation

When evaluating GAN performances, comparing the images might be irrelevant since the goal is **tricking** the Critic from the perspective of the Generator. That is exactly why we use GANS: to explore the feature space and find interesting combinations still accepted as real images. If there are unacceptable *generated* images, then one should fix the Critic/Discriminator by adding those images to the dataset with **fake** labels.

In [1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 72
import numpy as np
import sys

import torch
from torch import nn, optim
import torchvision

print(f'PyTorch version= {torch.__version__}')
print(f'torchvision version= {torchvision.__version__}')
```

```
print(f'CUDA available= {torch.cuda.is_available()}')

# Set the GPU to device 0
Device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

```
PyTorch version= 2.9.1+cu130
torchvision version= 0.24.1+cu130
CUDA available= True
```

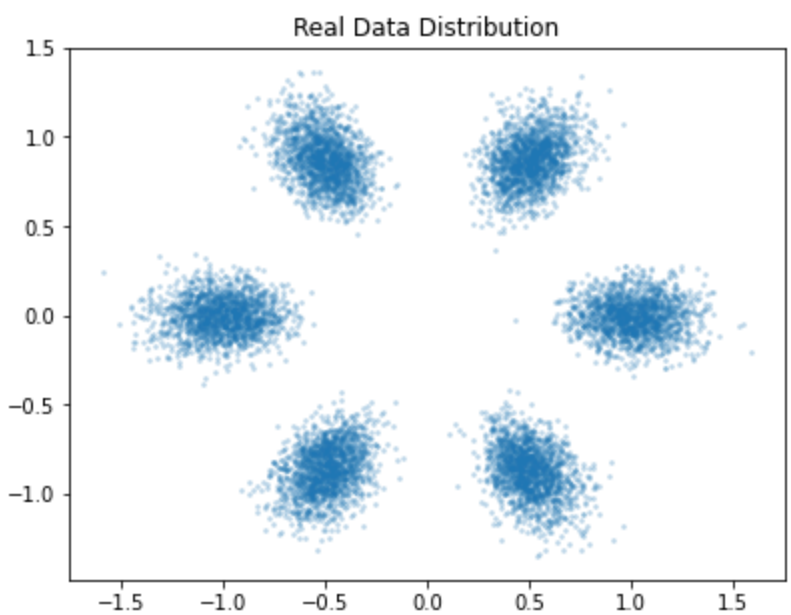
In [2]:

```
N_DATA= 10000

def synth_data(n=N_DATA, nclasses=6, var=0.10):
    angles = np.linspace(0, 2 * np.pi, nclasses, endpoint=False)  # angles
    centers = np.array([[np.cos(angle), np.sin(angle)] for angle in angles

    gdata = []
    for c, center in enumerate(centers):
        # generate points around the center
        points = np.random.normal(loc=center, scale=var, size=(n//nclasses
        norms = np.linalg.norm(points, axis=1, keepdims=True)
        points = points * np.abs(norms)**0.5  # stretch large values
        # labels = np.full((n/nclasses, 1), class_label)
        points = points.reshape(-1, 2).astype(np.float32)
        gdata += [np.hstack((points, ))]
    gdata = np.vstack(gdata)  # combine all classes
    return gdata

Realdata = synth_data()
plt.scatter(Realdata[:,0], Realdata[:,1], s=3, alpha=0.2)
plt.title("Real Data Distribution")
plt.show()
```



In [3]:

```
from torch.utils.data import TensorDataset, DataLoader

BATCH_SIZE = 1024
```

```
dset = TensorDataset(torch.tensor(Realdata))
Dloader_gan = DataLoader(dset, batch_size=BATCH_SIZE, shuffle=True,
                         num_workers=8, pin_memory=True, prefetch_factor=4
```

In [4]:

```
Z_DIM= 4

class Generator(nn.Module):
    def __init__(self, input_dim=Z_DIM, output_dim=1):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU(),
            nn.Linear(16, 32),
            nn.ReLU(),
            nn.Linear(32, output_dim)
        )
    def forward(self, _x):
        return self.model(_x)

class Discriminator(nn.Module):
    def __init__(self, input_dim=1):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.LeakyReLU(0.2),
            nn.Linear(32, 16),
            nn.LeakyReLU(0.2),
            nn.Linear(16, 1),
            nn.Sigmoid()
        )
    def forward(self, _x):
        return self.model(_x)

class Critic(nn.Module):
    def __init__(self, input_dim=1):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.LeakyReLU(0.2),
            nn.Linear(32, 16),
            nn.LeakyReLU(0.2),
            nn.Linear(16, 1)
            # no activation
        )
    def forward(self, _x):
        return self.model(_x)

class GAN:
    def __init__(self, epochs=100, eta=0.001, info=False):
        self.epochs, self.eta, self.info = epochs, eta, info
        self.criterion = nn.BCELoss()
        self.netD, self.netG, self.optimizerD, self.optimizerG = None, Non
        self.labels_real, self.labels_fake, self.labels_gen = None, None,
        self.LossDReal, self.LossDFake, self.LossG = None, None, None
```

```
        self.LabelReal = 1
        self.LabelFake = 0

    def initDG(self):  # 2 dimensional
        self.netD = Discriminator(input_dim=2).to(Device)
        self.netG = Generator(output_dim=2).to(Device)
        # Optimizers
        self.optimizerD = optim.Adam(self.netD.parameters(), lr=self.eta)
        self.optimizerG = optim.Adam(self.netG.parameters(), lr=self.eta)

    def execD(self, _x_real):
        bs = _x_real.size(0)
        self.netD.zero_grad()

        self.labels_real = torch.full((bs,1), self.LabelReal, dtype=torch.
        y_real = self.netD(_x_real)
        loss_D_real = self.criterion(y_real, self.labels_real)

        z_noise = torch.randn(bs, Z_DIM, device=Device)
        x_fake = self.netG(z_noise)
        self.labels_fake = torch.full((bs,1), self.LabelFake, dtype=torch.
        y_fake = self.netD(x_fake.detach())
        loss_D_fake = self.criterion(y_fake, self.labels_fake)

        loss_D = loss_D_real + loss_D_fake
        loss_D.backward()
        self.optimizerD.step()
        return x_fake, loss_D_real, loss_D_fake, bs

    def execG(self, _bs):
        self.netG.zero_grad()
        self.labels_gen = torch.full((_bs,1), self.LabelReal, dtype=torch.
        z_noise = torch.randn(_bs, Z_DIM, device=Device)
        x_fake = self.netG(z_noise)
        y_fake_r = self.netD(x_fake)
        loss_G = self.criterion(y_fake_r, self.labels_gen)
        loss_G.backward()
        self.optimizerG.step()
        return loss_G

    def fit(self, _train_dl):
        self.initDG()

        self.LossDReal, self.LossDFake, self.LossG = [], [], []

        for e in range(self.epochs):
            lossDr, lossDf, lossG = [], [], []
            for data in _train_dl:
                x_real = data[0].to(Device)

                _, loss_D_real, loss_D_fake, bs = self.execD(x_real)
                loss_G = self.execG(bs)

                if self.info:
                    sys.stderr.write("\r{:03d}/{:3d} | LossDr: {:6.5f} | l
```

```
                        e+1, self.epochs, loss_D_real.mean().item(), loss_
                    sys.stderr.flush()

                lossDr += [loss_D_real.mean().item()]
                lossDf += [loss_D_fake.mean().item()]
                lossG += [loss_G.mean().item()]

            self.LossDReal += [(np.array(lossDr).mean(), np.array(lossDr).
            self.LossDFake += [(np.array(lossDf).mean(), np.array(lossDf).
            self.LossG += [(np.array(lossG).mean(), np.array(lossG).std())

class WassersteinLoss(nn.Module):
    def __init__(self):
        super().__init__()
    @classmethod
    def forward(cls, _input, _target):  # input is prediction, target is l
        if _target is None:  # shortcut using 1 and -1
            return -torch.mean(_input)
        return -torch.mean(_input*_target)

class WGAN(GAN):
    def __init__(self, epochs=100, eta=0.001, info=False, n_critic=5, clip
        super().__init__(epochs, eta, info)
        self.n_critic, self.clipw = n_critic, clipw
        self.criterion = WassersteinLoss()
        self.LabelFake = -1

    def initDG(self):
        self.netD = Critic(input_dim=2).to(Device)
        self.netG = Generator(output_dim=2).to(Device)
        self.optimizerD = optim.RMSprop(self.netD.parameters(), lr=self.et
        self.optimizerG = optim.RMSprop(self.netG.parameters(), lr=self.et

    def execD(self, _x_real):
        bs = _x_real.size(0)
        x_fake, loss_D_real, loss_D_fake = None, None, None
        for _ in range(self.n_critic):
            self.netD.zero_grad()
            y_real = self.netD(_x_real)
            loss_D_real = self.criterion(y_real, None)

            z_noise = torch.randn(bs, Z_DIM, device=Device)
            x_fake = self.netG(z_noise)
            y_fake = self.netD(x_fake.detach())
            loss_D_fake = -self.criterion(y_fake, None)  # careful with co

            loss_D = loss_D_real + loss_D_fake
            loss_D.backward()
            self.optimizerD.step()

            for p in self.netD.parameters():
                p.data.clamp_(-self.clipw, self.clipw)

        return x_fake, loss_D_real, loss_D_fake, bs
```

```
    def execG(self, _bs):
        self.netG.zero_grad()
        z_noise = torch.randn(_bs, Z_DIM, device=Device)
        x_fake = self.netG(z_noise)
        y_fake_r = self.netD(x_fake)  # use fresh x_fake
        loss_G = self.criterion(y_fake_r, None)
        loss_G.backward()
        self.optimizerG.step()
        return loss_G
```

In [5]:
```
def gen_plot(_ax, _netG):
    _netG.eval()
    with torch.no_grad():
        z_noise = torch.randn(N_DATA, Z_DIM, device=Device)
        gendata = _netG(z_noise).cpu().numpy()
    _ax.scatter(gendata[:,0], gendata[:,1], s=3, alpha=0.2)
```

In [6]:
```
%%time

gan = GAN(epochs=3000, eta=0.001, info=True)
gan.fit(Dloader_gan)
```

```
3000/3000 | LossDr: 0.72763 | lossDf: 0.66621 | lossG: 0.72221
CPU times: total: 2min 12s
Wall time: 2min 15s
```

In [7]:
```
%%time

wgan = WGAN(epochs=3000, eta=0.001, info=True, n_critic=5)
wgan.fit(Dloader_gan)
```

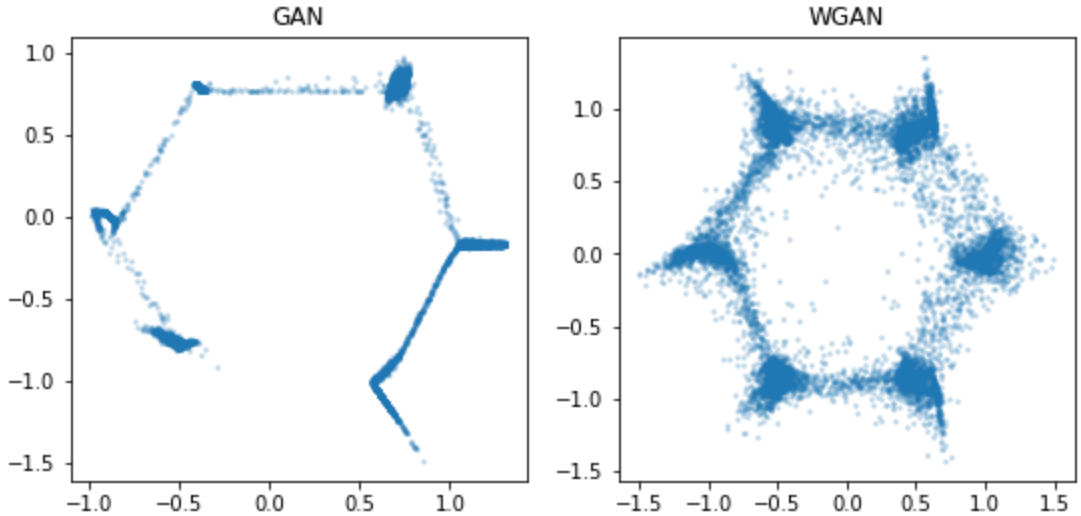
```
3000/3000 | LossDr: -0.00924 | lossDf: 0.00860 | lossG: -0.007829
CPU times: total: 7min 4s
Wall time: 7min 23s
```

In [8]:
```
_, ax = plt.subplots(1, 2, figsize=(9,4))

gen_plot(ax[0], gan.netG)
ax[0].set(title="GAN")

gen_plot(ax[1], wgan.netG)
ax[1].set(title="WGAN")

plt.show()
```



In [9]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```